



Timing Attacks on RSA and Fermat's factorization

Michel Jean Joseph Donnet

Faculty of Science, University of Geneva

2026-05-19



Outline

Introduction

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Bibliography

Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Cryptography

Goals

- Confidentiality
- Integrity
- Authenticity

Types of cryptography

- Symmetric cryptography (e.g., AES)
- Asymmetric cryptography (e.g., RSA)

Security of cryptographic algorithms

- Computational hardness assumptions
- Mathematical problems (e.g., factoring large integers for RSA)

RSA algorithm

Security assumption

- Factorization of large integers is computationally hard

Key generation

- p, q : large prime numbers
- $n = p \times q$: modulus
- e : public exponent, coprime to $\varphi(n)$
- d : private exponent, such that $e \cdot d \equiv 1 \pmod{\varphi(n)}$

Keys

- Public key: (n, e)
- Private key: (n, d)

RSA algorithm

Encryption

- Plaintext message m (integer less than n)
- Ciphertext $c = m^e \bmod n$

Decryption

- Ciphertext c
- Plaintext message $m = c^d \bmod n$

Proof

$$c^d \bmod n = (m^e)^d \bmod n = m^{ed} \bmod n = m^{1+k\varphi(n)} \bmod n = m \bmod n$$



Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

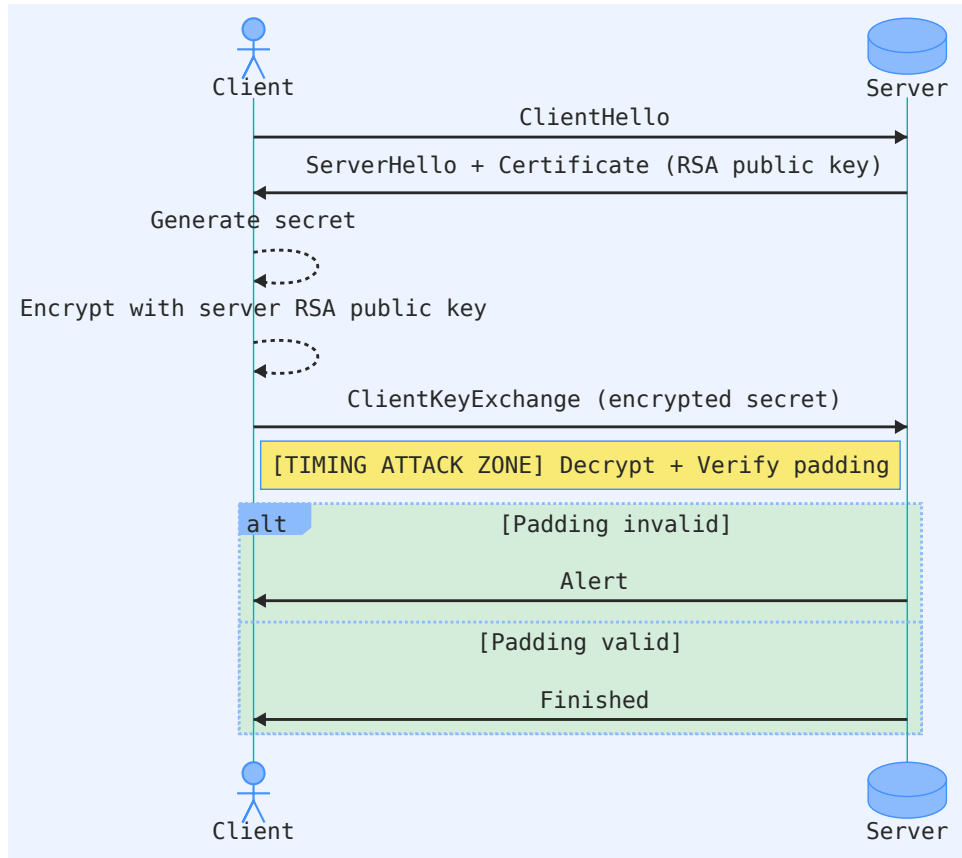
Challenges in implementation

Results

Fermat's factorization

Conclusion

Example Protocol



Square-and-multiply algorithm

The execution time can vary based on the value of the exponent and the input ciphertext, as explained by Paul C. Kocher [1].

```
def square_and_multiply(y, x, n):  
    s = 1  
    y %= n  
  
    while x > 0:  
        if (x % 2) == 1:  
            s = (s * y) % n # Extra multiplication here when x is odd !  
        y = (y * y) % n  
        x = x >> 1  
  
    return s
```

Exploiting timing variations

Online

1. Attacker sends random ciphertexts c to the server
2. Measures time for decryption $c^d \bmod n$

Offline

1. For each bit of d , guess bit value (0 or 1)
2. Compute expected timing based on guess
3. Analyze variance of timing differences
 - Low variance: guess likely correct
 - High variance: guess likely incorrect

Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Mathematical analysis

Probability of a correct guess

$$P(x_b) \propto \prod_{i=1}^N F(T_i - t(c_i, x_b) \mid x_b)$$

with F : probability distribution of timing error according to x_b .

$$P(x_b = 1) = \frac{\prod_{i=1}^N F(T_i - t(c_i, x_b) \mid x_b = 1)}{\prod_{i=1}^N F(T_i - t(c_i, x_b) \mid x_b = 0) + \prod_{i=1}^N F(T_i - t(c_i, x_b) \mid x_b = 1)}$$

- $P(x_b = 1) > 0.5 \implies x_b = 1$
- $P(x_b = 1) < 0.5 \implies x_b = 0$

Variance analysis

Assumptions

- $T_i = \sum_{k=1}^l t_k + \varepsilon$
- x_{b-1} is known

Case with correct guess x_b

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^l t_k + \varepsilon$$

Case with incorrect guess x_b

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^l t_k + \varepsilon + (t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}})$$

Variance analysis

Impact of previous errors on the variance

- The c -th bit of d is incorrect
- $c < b - 1$

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^l t_k + \varepsilon + \left(\sum_{k=c}^b (t_{k_{\text{correct}}} - t_{k_{\text{incorrect}}}) \right)$$



Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Pearson correlation coefficient

Definition

Measure of the linear correlation between two variables X and Y

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

- r close to 1: strong positive correlation
- r close to -1 : strong negative correlation
- r close to 0: no linear correlation



Designing the model of expected timings

Goal

Design a model that explains variations in execution time.

Model

Hamming weight of intermediate values in square-and-multiply algorithm

Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Sources of noise

- CPU cache effects
- Branch prediction
- Garbage collection
- Multiplication optimizations
- Other optimizations (e.g., loop unrolling, instruction reordering, just-in-time compilation)

Reducing the impact of noise

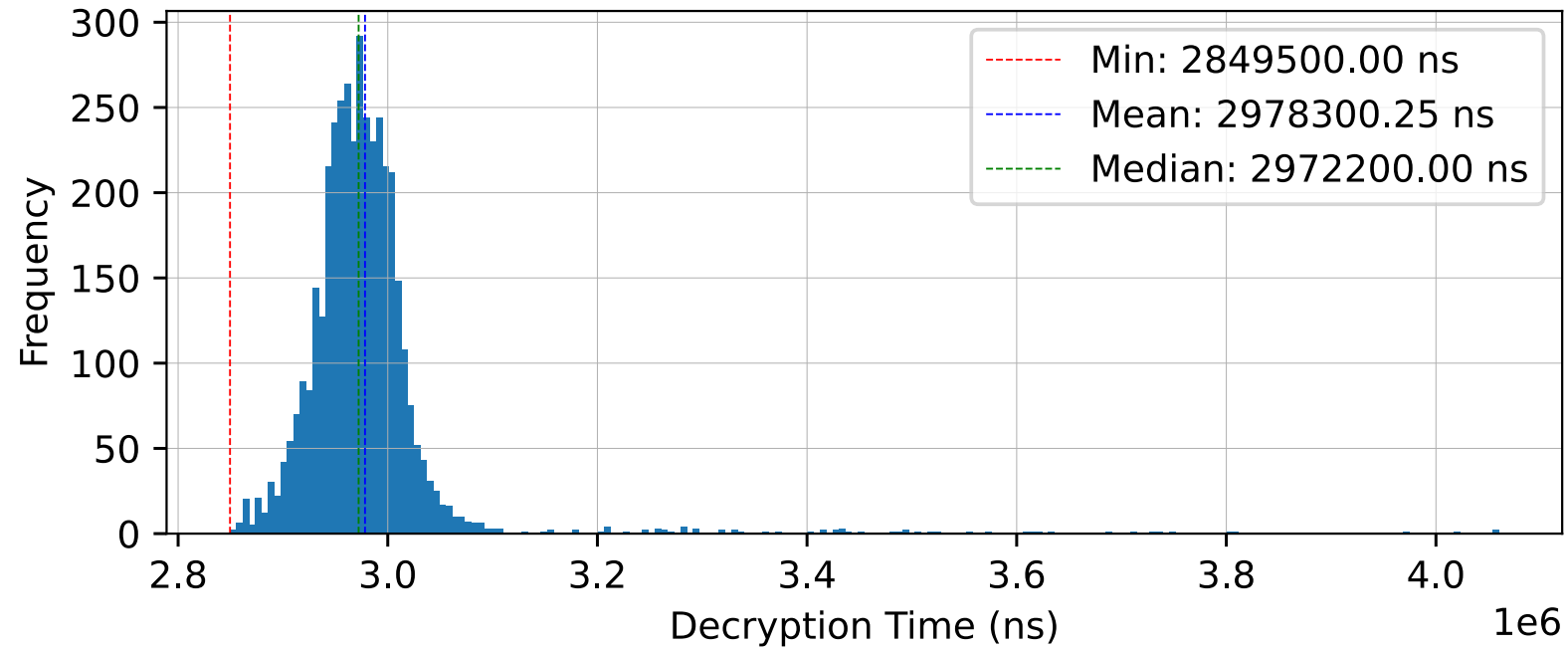


Figure 1: Distribution of timing measurements

Reducing the impact of noise

Garbage collector

Disable garbage collector during timing measurements [2]

Measurement repetitions

Using repeated measurements to reduce noise

Important

- Noise is additive
- High variance in timing measurements

⇒ **MINIMUM AS ESTIMATOR !**

Outline

Introduction

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Bibliography

Variance-based timing attack

Implementation

- Python without noise reduction
 - Failed
- Python with noise reduction and average as estimator
 - Failed
- C++ and Rust with average
 - Failed
- Python using **minimum** as estimator
 - Success

Variance-based timing attack

Results

- Recovered 5 bits of private key d
- Time taken: **13 hours** on AMD Ryzen 9 7950X3D
- Configuration:
 - 1000 messages
 - 10000 repetitions
 - Assumed first 16 bits of d known
 - 3 candidates per iteration

Pearson-based timing attack

Using median as estimator

- Recovered **7/10 bits** of private key d
- Time taken: **1h30**
- Configuration:
 - 3000 messages
 - 500 repetitions
 - Assumed first 16 bits of d known
 - 3 candidates per iteration

Using minimum as estimator

- Recovered **9/10 bits** of private key d
- Time taken: **1h30**



Outline

Introduction

Bibliography

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Factorization of the modulus n

Requirements

- $n = p \times q$
- p and q are odd primes and close to each other

Theory

- p and q are odd primes $\implies$ there exists an integer x such that $x = \frac{p+q}{2}$
- $y = x - p = q - x$
- $n = p \times q = (x - y)(x + y) = x^2 - y^2$

Factorization of the modulus n

Method [3]

1. $x = \lceil \sqrt{n} \rceil$
2. $y^2 = x^2 - n$
3. If y^2 is a perfect square
 - $y = \sqrt{y^2}$
 - $p = x - y$
 - $q = x + y$
4. Otherwise, increment x and repeat from step 2

Results

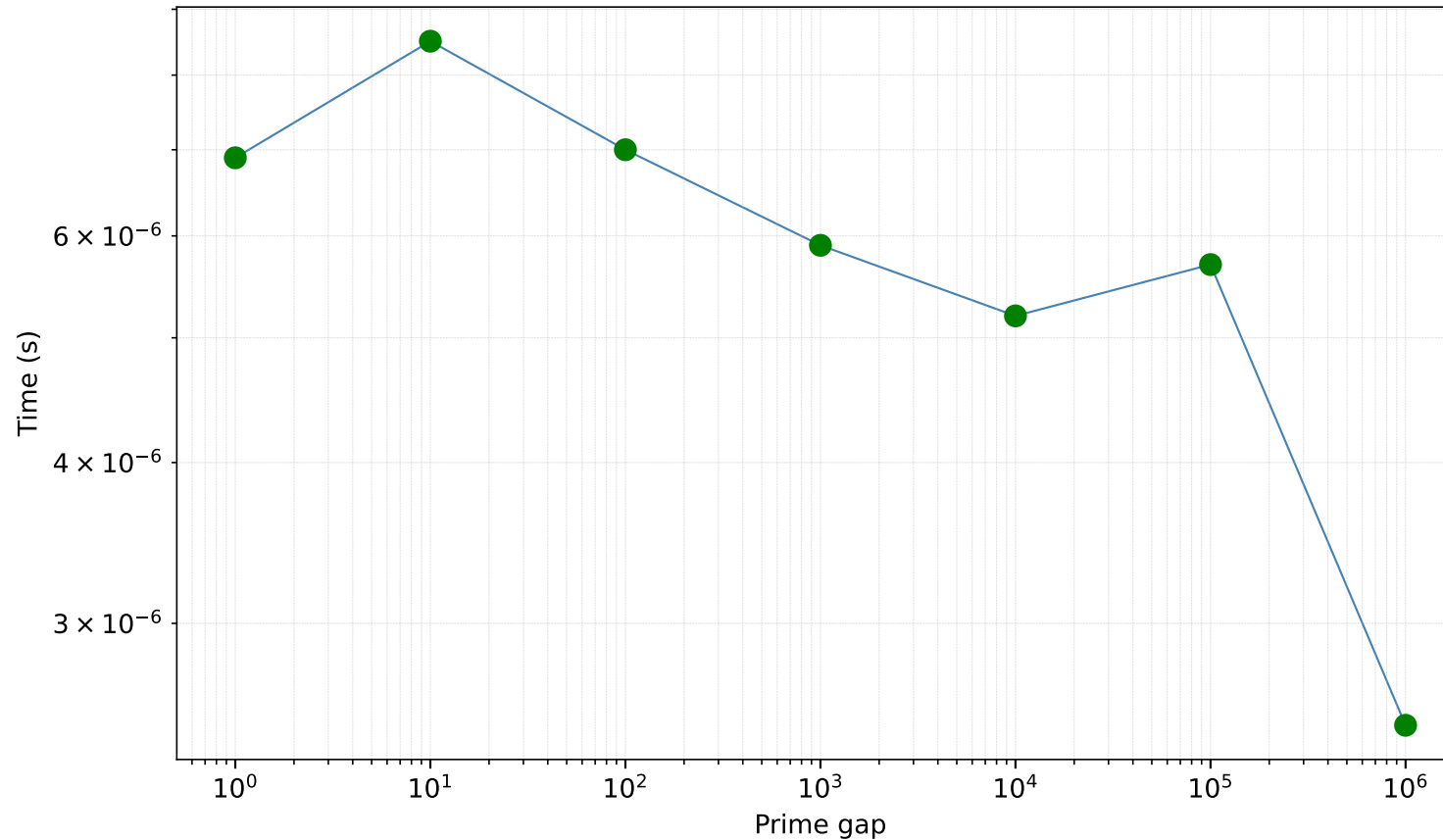


Figure 2: Fermat factorization with n: 2048 bit

Outline

Introduction

Principles of RSA timing attacks

Mathematical analysis

Pearson-based timing attack

Challenges in implementation

Results

Fermat's factorization

Conclusion

Bibliography

Conclusion

Algorithms can be **mathematically secure** but still **vulnerable** to side-channel attacks that exploit **physical implementations** of the algorithms, such as timing variations in RSA decryption.

Timing attacks are **still relevant** today.

They highlight the importance of **considering** side-channel attacks when **designing** and **implementing** cryptographic systems.

Bibliography

Bibliography

- [1] P. C. Kocher, “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems,” in *Annual international cryptology conference*, 1996, pp. 104–113.
- [2] M. Ochoa, “Kocher's Timing Attack: A Journey from Theory to Practice.” Accessed: May 10, 2026. [Online]. Available: <https://blog.zksecurity.xyz/posts/timing-kocher/>
- [3] H. Böck, “Fermat Factorization in the Wild.” [Online]. Available: <https://eprint.iacr.org/2023/026>